A Taste of the Snark
Mathcamp Week 5

Antonio Kambiré

1 Day 1

1.1 Logistics

Come to TAU, and we will discuss fun, hopefully enlightening examples of any of the concepts we will discuss in class. On day 2, we might even look at real-life Groth'16 provers. Also, there are certainly more rigorous definitions for all the concepts discussed here, so also come to TAU if you want a more detailed or rigorous treatment.

1.2 Traditional Proofs

When we think about traditional proofs, for example in mathematics, the image of a long text comes to mind. These take linear time in the size of the text to verify, as you need to verify the validity of every step of the proof one after the other.

In logic or complexity theory, we can think of problems that require proofs as relations. **A relation** R is a subset of $X \times W$ where $X \subseteq \{0,1\}^*$ is a set containing statements to be proven and $W \subseteq \{0,1\}^*$ is a set containing proofs, also called witnesses, for these statements.

Languages: For a relation R, we can define $L(R) = \{x \mid \exists w : (x,w) \in R\}$. These represent the set of "true" statements.

Example: An example of a relation is:

$$R_{PQ} = \{(N,(p,q)) \mid p,q \text{ are prime and } N = pq\}$$

This encodes the problem of showing that an integer N is the product of two primes.

We say that a relation R is an NP-relation if there exists a poly-time algorithm $\mathcal{V}$ (the time taken by the algorithm is bounded by a polynomial of the size of the input), called a verifier, such that:

$$\mathcal{V}(x,w) = 1 \iff (x,w) \in R$$

1.3 Interactive Proofs

Proof system: From the above, we can start to conceive of the idea of a proof system for a certain relation R as an interaction between two actors, a prover $\mathcal{P}$ and a verifier $\mathcal{V}$. The verifier and the prover are both given a statement x and the prover needs to convince the verifier that they know a witness w such that $(x,w) \in R$. We will define a proof system by the triplet $(\mathcal{P},\mathcal{V},R)$. We note by $Out(\mathcal{P},\mathcal{V},x)$ the output of one interaction between prover $\mathcal{P}$ and verifier $\mathcal{V}$ about a statement x.

Normal proof systems only have one round of interaction where the prover just sends w to the verifier. For example, if I wanted to prove to you that $N \in R_{PQ}$, all I need to do is to send you p,q and you can check yourself that they are in the relation.

The exciting thing with interactive proofs is that it is possible, using more interactivity, to gain incredible properties for our proof systems. This forces our proof system to become probabilistic. Indeed, if the verifier was deterministic, there would be no need for interaction at all as the prover could simulate the verifier internally. For this class, we will be working in a public coin setting where both the verifier and the prover have access to the same source of randomness.

However, like with all nice things, gaining all these cool properties will come with a tradeoff. In particular, we would be losing a bit of accuracy. What does accuracy mean? There are two things that we want any proof system to get right:

1. **Completeness:** We want the verifier to always accept true statements. So for all true statements x and honest $\mathcal{P}$, we want $\Pr[Out(\mathcal{V},\mathcal{P},x) = 1] = 1$.

2. **Soundness:** We want the verifier to reject false statements. This is where we allow some margin of error. In particular, we say that a proof system has soundness error ε if for all false statements

x' and a dishonest $\mathcal{P}'$, we want:

$$\Pr[Out(\mathcal{V}, \mathcal{P}', x') = 1] = \varepsilon$$

We say that the proof system is computationally sound if the soundness error is negligible against probabilistic-poly-time (which we will note ppt.) provers. Our proof system is now called an argument system instead.

3. **Knowledge soundness:** On top of regular soundness, we would like our argument system to exhibit a property called knowledge soundness, which says that if $\mathcal{P}$ succeeds to convince $\mathcal{V}$, then it must *know* a witness w. In the example above of R_{PQ}, it may be fine in most cases to just be convinced that N can be written as pq, but we may also want to be sure that the person we are dealing with *knows* the factors p and q. This is formalized through the existence of an algorithm $\mathcal{E}$ called an extractor, in the non-black-box model of computation such that $\mathcal{E}$ can output the witness when given special access to $\mathcal{P}$'s internal state. This is a tricky topic to understand so we will not go too deeply in it. Our new system is now called an argument of knowledge. How sweet!

At the cost of this negligible soundness error, we gain a lot! In particular, we can construct an interactive argument system for all NP relations that fulfills:

1. **Succinctness:** The total transcript of the communication between prover and verifier will be sublinear in the size of w. In fact we will be arriving at a constant size "proof"!

2. **Zero-Knowledge:** The verifier doesn't learn anything at all about the witness through its interaction with the prover. We formalize by assuming that $\mathcal{V}$ cannot learn anything new from an interaction if it could have simulated the interaction on its own. In particular, a proof system is Honest Verifier Zero Knowledge (HVZK) if there exists an ppt. algorithm $\mathcal{S}$, called a simulator such that for all $(x, w) \in R$:

$$\{\mathcal{S}(x)\} \sim \{tr(P, V, x)\}$$

where $tr(\mathcal{P}, \mathcal{V}, x)$ is the transcript of the interaction between honest P and V for statement x. The symbol $\sim$ can represent perfect, statistical, or computational indistinguishability depending on how resilient of a system we need. Here we are specifically assuming that the verifier follows the protocol correctly, which is why we have the Honest Verifier prefix. However, this is not much of an issue because most schemes that provide HVZK can also be proven fully ZK.

In this class, as advertised, we will are trying to build a zk-SNARK: A zero-knowledge succinct non-interactive argument of knowledge. What? Non-interactive? Isn't the whole point of all of this that interactivity is necessary to provide the other properties? The answer is, not really. The key to this sorcery lies in the idea of succinct polynomial commitments.

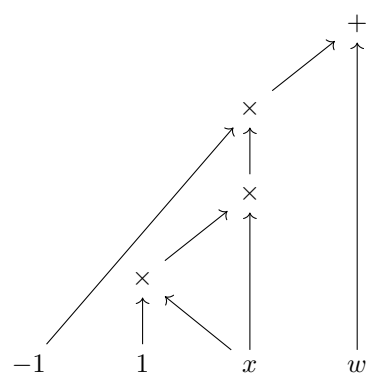
1.4 Finding a 'Nice' NP representation

Arithmetic circuit: Arithmetic circuits are 2 fan-in circuits which, instead of using logic gates, have addition (+) and multiplication (×) in a large field $\mathbb{F}_p$ where p is prime. The circuit will have x and w as input in the bottom layer and will have a single output gate at the top layer. Under this representation, every NP relation can be written as:

$$R = \{(x, w) \mid C_R(x, w) = 0\}$$

where C_R is an arithmetic circuit.

Here is an example of a simple arithmetic circuit that checks whether $x^2 = w$.



The public inputs here are made of $(-1, 1, x)$ and the private inputs are the bits of w.

Constraint system: Every arithmetic circuit C can be written as a system of constraints, each of which representing the equality condition of a gate (there are ways to compactify this but it's not necessary) such that if each constraint is satisfied, then the circuit outputs 0. For example the circuit above

can be written as a system of constraints as:

$$\begin{cases} c_0 = x \times 1 \\ c_1 = x \times c_0 \\ c_2 = -1 \times c_1 \\ c_3 = c_2 \times w \\ c_3 = 0 \end{cases}$$

Which can be compactified into:

$$\begin{cases} c_0 = w - x \times x \\ c_0 = 0 \end{cases}$$

R1CS: Given a constraint system like the one above, we can rewrite each constraint as a rank-1 constraint system (R1CS) of the form:

$$(s \cdot a) \times (s \cdot b) = s \cdot c$$

where s represents the vector (x, w, c) of public and private inputs as well as intermediate values and $\circ$ represents component-wise multiplication. For example, for the constraint system above, if $s = \big[1, x, w, c_0\big]$ then the first constraint can be written as:

$$s \cdot \big[0, 1, 0, 0\big] \times s \cdot \big[0, 1, 0, 0\big] = s \cdot \big[0, 0, 1, 1\big]$$

The second constraint will be written as:

$$s \cdot \big[1, 0, 0, 0\big] \times s \cdot \big[0, 0, 0, 1\big] = s \cdot \big[0, 0, 0, 0\big]$$

Given any circuit, we can use a similar procedure to turn it into a system of R1CS. Therefore a system of R1CS is enough to represent all of NP.

1.5 Quadratic Arithmetic Programs

Given a system of R1CS, we can turn it into an equivalent form that uses polynomials instead. First, we take all of our constraints and put them into three matrices. For example, for the R1CS above:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} B = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} C = \begin{bmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

All our constraints now become:

$$(A \cdot s) \circ (B \cdot s) = C \cdot s \iff (A \cdot s) \circ (B \cdot s) - (C \cdot s) = 0$$

There are two remarkable facts of polynomials that will allow us to reduce this into a very convenient form:

1. **Lagrange Interpolation:** We are going to construct a polynomial for each of our matrices (to get something like $A_c(x), B_c(x)$ and $C_c(x)$ for c a column) such that for cell (r, c), we have $A_c(r) = A_{r,c}$. This can be done efficiently (in time $O(n \log n)$) using Lagrange Interpolation and the Fast Fourier Transform. Come see me at TAU and I'll explain!

   After making each column of our three matrices into a polynomial, we get:

$$\begin{array}{c} s \cdot \begin{bmatrix} A_1(1) & A_2(1) & \cdots & A_m(1) \\ A_1(2) & A_2(2) & \cdots & A_m(2) \\ \vdots & \vdots & \ddots & \vdots \\ A_1(n) & A_2(n) & \cdots & A_m(n) \end{bmatrix} \\ \circ \\ s \cdot \begin{bmatrix} B_1(1) & B_2(1) & \cdots & B_m(1) \\ B_1(2) & B_2(2) & \cdots & B_m(2) \\ \vdots & \vdots & \ddots & \vdots \\ B_1(n) & B_2(n) & \cdots & B_m(n) \end{bmatrix} \\ - \\ s \cdot \begin{bmatrix} C_1(1) & C_2(1) & \cdots & C_m(1) \\ C_1(2) & C_2(2) & \cdots & C_m(2) \\ \vdots & \vdots & \ddots & \vdots \\ C_1(n) & C_2(n) & \cdots & C_m(n) \end{bmatrix} \\ = 0 \end{array}$$

   Checking row by row, this is the same as

$$(\sum_{i=1}^{m} s_i A_i(x))(\sum_{i=1}^{m} s_i B_i(x)) - (\sum_{i=1}^{m} s_i C_i(x)) = 0$$

   for $x = 1, 2, \ldots, m$.

2. **Polynomial roots:** We can convince ourselves that a polynomial P is 0 at i iff there exists some polynomial $H(x)$ such that $P = H(x - i)$.

   We note $Z(x) = (x - 1)(x - 2) \ldots (x - m)$ and we can now reduce the whole problem into the existence of some low degree polynomial H such that:

$$A(x)B(x) - C(x) = H(x)Z(x)$$

   In this form, our problem is now called a Quadratic Arithmetic Program. The prover needs to give us H, we can plug it in ourselves and check whether the circuit is indeed satisfied. It is important that H is low degree ($< n - 1$). Our work tomorrow will be to use a Polynomial Commitment Scheme to actually arrive at a powerful zk-SNARK.

1.6 Homework Day 1

1.6.1 On Interpolation

Given a list of points $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$ such that $i \neq j \implies x_i \neq x_j$, find a way to construct a polynomial $P(x)$ of degree $< n$ such that $P(x_i) = y_i$. What is the runtime of your algorithm in terms of n? Can you make it faster? If you couldn't find something that works, this is okay! It's your reminder to go read about Lagrange Interpolation :)

1.6.2 An R1CS for Range Check

(Borrowed from Stanford CS355)

Let $\mathbb{F}_p$ be a prime finite field, where $p >> 2^\ell$. Design an R1CS program $A, B, C \in \mathbb{F}_p^{k \times m}$ that accepts exactly the set of elements $\{0, 1, \cdots, 2^\ell - 1\} \subset \mathbb{F}_p$. More precisely, your R1CS program A, B, C takes only one public input $x_1 \in \mathbb{F}_p$, and the language accepted by the program is $L(R_{A,B,C}) = \{0, 1, \cdots, 2^\ell - 1\}$. Your program should have at most $\ell + 1$ constraints.

Hint: Observe that $x \in \mathbb{F}_p$ satisfies $0 \leq x < 2^\ell$ if and only if the binary representation of x has at most ℓ bits. Try to provide the bits in the binary representation of x as the witness to your R1CS program.

1.6.3 The Schnorr Protocol

The Schnorr protocol is a protocol that allows one to prove knowledge of the discrete log of an element in a group. More specifically, given the public inputs of $g \in \mathbb{G}$ the generator of the group, and $h \in \mathbb{G}$, the prover is trying to convince the verifier that they know x such that $h = g^x \in \mathbb{G}$.

It goes like this:

Prover		Verifier
$r \xleftarrow{\$} \mathbb{Z}_q^*$ $u = g^r$		
	$\xrightarrow{u}$	
		$\mathsf{schnorr_validate}(u, h)$ – – – – – – – $c \xleftarrow{\$} \mathbb{Z}_q$
	$\xleftarrow{c}$	
$z = r + x \cdot c$		
	$\xrightarrow{z}$	
		$z \neq 0 \mod q$ – – – – – – – $g^z \stackrel{?}{=} u \cdot h^c$

schnorr_validate is a function that checks that $u \in \mathbb{G}$ and that $h, u \neq 0$.

Your exercise consists of:

1. Showing that this proof system is complete.
2. Finding the soundness error.
3. Constructing a simulator for the protocol, showing that it's zero-knowledge.

1.6.4 The Fiat-Shamir Transform

Note: This problem assumes understanding of hash functions.

Before we get into this problem, you may want to know that NP has been shown to be equal to a class of interactive protocols called Σ protocols which have only 2 rounds (back and forths) of communication:

1. The prover sends a commitment c
2. The verifier sends with a challenge r (generally sampled uniformly at random)
3. The prover responds with a response a
4. The verifier either accepts or rejects

The Schnorr protocol from the last problem is an example of Σ protocol.

The Fiat-Shamir transform uses the collision-resistance property of hash functions to compile a non-interactive version of Σ protocols. In particular, instead of requesting r from the verifier, the prover will instead use a hash of all public values up to that point in the interaction. In the case of the Schnorr protocol, this is something that looks like $r = \mathbf{Hash}(p, g, h, c)$ where p is the order of $\mathbb{G}$.

Strong collision resistance: A family of hash functions G is collision resistant if for all ppt if

$$\Pr[h \leftarrow G, (x_1, x_2) \leftarrow A(h) \mid x_1 \neq x_2, h(x_1) = h(x_2)]$$

is negligible. This property basically says that it's hard to find two different inputs with the same hash.

This problem asks you to do the following:

1. Construct the non-interactive Schnorr protocol.
2. Assuming strong collision resistance for a family of hash functions G, prove that the non-interactive version Schnorr protocol is still computationally sound.
3. Can you find an attack to the non-interactive Schnorr protocol if instead of using $r = \mathbf{Hash}(p, g, h, c)$, we use $r = \mathbf{Hash}(p, g, c)$?

2 Day 2

2.1 The Schwartz-Zippel Lemma

Last time, we showed that all of NP can be represented as a QAP of the form $A(x)B(x) - C(x) = H(x)Z(x)$. We want to find a way to efficiently check this equality. Notice that the equality can be rewritten as:

$$P(x) = A(x)B(x) - C(x) - H(x)Z(x) = 0$$

So our work now consists in checking that the left hand size is identically 0. For this, we use the following lemma:

Schwartz-Zippel: Let $P \in R[x_1, x_2, \ldots, x_n]$ be a non-zero polynomial of total degree $d \geq 0$ (the maximum of the degree of each monomial, which is the sum of the powers in that monomial) over an integral domain R. Let S be a finite subset of R. If $r_1, r_2, \ldots, r_n \leftarrow S$, then:

$$\Pr[P(r_1, r_2, \cdots, r_n) = 0] \leq \frac{d}{|S|}$$

In our case, we have a univariate polynomial P, such that:

$$\Pr[P(x) = 0] \leq \frac{d}{|S|} \leq \frac{2(n-1)}{p} \approx 0$$

This can be proved pretty easily through induction on n using the Fundamental Theorem of Algebra to prove the base case.

This is pretty neat! This lemma is saying that if I am given a non-zero polynomial, the probability that it is 0 at a random point is very low. We can use this to our advantage: Instead of actually checking that the coefficients of the polynomial above all cancel, which is computationally expensive, we are going to evaluate the polynomial at a random point and we have a very high probability that it will be 0 only if $P(x) = 0$.

There is a little bit of a dilemma here, where the verifier wants to be able to evaluate the polynomials detained by the prover at a random point but the prover doesn't want to allow us to access its polynomials because this would break zero-knowledge. On the other hand, the verifier cannot just give the random point to the prover to evaluate because this would be a bit silly and would break soundness.

2.2 Polynomial Evaluations

First, let's talk about hardness assumptions. Remember that we are ultimately trying to prove that our proof system is secure only for bounded provers (this is why we have an *argument* system in fact. In cryptography, people use problems that are known to be hard to solve as tools to construct systems that are computationally secure, as described above.

One such problem that we assume is very hard to solve (heuristically) is the discrete log problem. A lot of research has gone into solving this problem for decades now and no true progress has been made (except if we use quantum computers), so it is often use as a safe tool in encryption protocols. This problem is set in a suitable cyclic group $\mathbb{G}$ (for example the group of rational points on good elliptic curves) of prime order p with random generator g and is represented by the following relation:

$$R_{DLOG} = \{((g, h), x) \mid h = g^x\}$$

From now on, for ease of notation, I will be noting $[x] = g^x$

We first notice that to evaluate a polynomial at a random point t, all one needs are the powers of t: $t^1, t^2, \ldots, t^n$.

With this in mind, here is one way to solve the problem of polynomial evaluation problem: We pick some random number α, and we compute $[\alpha], [\alpha t], [\alpha t^2], \ldots, [\alpha t^n]$ and send it to the prover. Note that this means that the prover can only compute linear combinations of αt^i's. Given a polynomial $P(x) = \sum_{i=0}^{n} p_i t^i$, the prover can compute $[\alpha P(t)] = [\alpha \sum_{i=0}^{n} p_i t^i] = \sum_{i=0}^{n} p_i [\alpha t^i]$ and send it back to the verifier. This one can then compute $\frac{1}{\alpha}[\alpha P(t)] = [P(t)]$. At the end, the verifier has an encryption of the evaluation of P at t.

This is pretty cool! However, it is still not enough to solve our QAPs. Indeed, we just *committed* to the polynomial but there's still no way for the verifier to actually check anything on $[P(t)]$. We will resolve this issue through the use of bilinear pairings.

2.3 Bilinear Pairings

A bilinear pairing, often called an elliptic curve pairing gives us a way to map two elements of two different groups to get an element of a third group. More specifically, it will be a map $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_t$ such that:

- $e(P, Q + R) = e(P, Q) \cdot e(P, R)$
- $e(P + Q, R) = e(P, R) \cdot e(Q, R)$

We will be dealing with pairings that are efficiently computable and non degenerate (that is, $e(P, Q) \neq 1$ for all P, Q). Since we will be working in two different groups, we will be noting $[a]_1 = g_1^a, [b]_2 = g_2^b$.

Bilinear pairings allow us to perform multiplicative checks on encrypted data. Given $[a]_1, [b]_2$, if someone claims that they know $[c]_1$ such that $ab = c$, I can check whether $e([a]_1, [b]_2) \stackrel{?}{=} e([c]_1, g_2)$ since $e([a]_1, [b]_2) = e(g_1, g_2)^{ab}$ and $e([c]_1, g_2) = e(g_1, g_2)^c$.

2.4 Groth'16 Step 1: Trusted setup

We are at last ready to explain the protocol for Groth'16. It all starts with a step called trusted setup. This step will generate encrypted values called the common reference string (crs) or the structured referenced string (srs) such that neither the verifier nor the prover will know the original values. In practice, this is either done on an enclave in a computer that both parties trust, or through a multi-party computation (MPC) involving both the verifier and the prover. More on this at TAU if interested.

Assuming that there are m inputs in total, the first k of which public values, and the rest is the witness, the set up steps are as follows:

1. Random values τ, α, β are generated
2. Phase 1: Specific powers of tau are computed:

$$[\tau^0]_1, [\tau^1]_1, [\tau^2]_1, \ldots, [\tau^{2n-2}]_1$$
$$[\tau^0]_2, [\tau^1]_2, [\tau^2]_2, \ldots, [\tau^{n-1}]_2$$
$$\alpha \cdot ([\tau^0]_1, [\tau^1]_1, [\tau^2]_1, \ldots, [\tau^{n-1}]_1)$$
$$\beta \cdot ([\tau^0]_1, [\tau^1]_1, [\tau^2]_1, \ldots, [\tau^{n-1}]_1)$$
$$\beta$$

3. Random values γ, δ are generated
4. Phase 2: Using the polynomials $A_i(x), B_i(x), C_i(x)$, we define the following polynomials:

$$L_i(x) = \beta A_i(x) + \alpha B_i(x) + C_i(x)$$

One useful fact about these polynomials is that they can be written as:

$$L_i(x) = (\alpha + A_i(x))(\beta + B_i(x)) - \alpha\beta + A_i(x)B_i(x)$$

We create the **proving key** made of:

$$\alpha, \beta, \delta$$
$$[\tau^0]_1, [\tau^1]_1, [\tau^2]_1, \ldots, [\tau^{n-1}]_1$$
$$\delta^{-1}([L_{k+1}(\tau)]_1, [L_{k+1}(\tau)]_1, \ldots, [L_m(\tau)]_1)$$
$$\delta^{-1}([\tau^0]_1, [\tau^1]_1, [\tau^2]_1, \ldots, [\tau^{n-2}]_1) \cdot Z(\tau)$$
$$[\beta]_2, [\delta]_2$$
$$[\tau^0]_2, [\tau^1]_2, [\tau^2]_2, \ldots, [\tau^{n-1}]_2$$

And the **verifying key**:

$$[\alpha]_1$$
$$\gamma^{-1}([L_1(\tau)]_1, [L_1(\tau)]_1, \ldots, [L_k(\tau)]_1)$$
$$[\beta]_2, [\gamma]_2, [\delta]_2$$

5. We destroy $\alpha, \beta, \delta, \gamma$. These are called toxic waste, and leaking either of these can be proven detrimental to soundness or zero-knowledge.

One helpful way to think about the trusted setup process is as generation of a good and compact description for our circuit, such that the verifier doesn't even need to refer to the original problem anymore to verify the validity of proofs. This is how we "cheat" and reach non-interactivity.

2.5 Groth'16 Step 2: Proving

Upon receiving the proving key, the prover, having access to the witness values $w_1, \ldots, w_m$, can compute the private witness polynomial:

$$L_w(x) = \sum_{i=k+1}^{m} w_i \cdot L_i(x)$$

Similarly, we can define the public input polynomial:

$$L_p(x) = \sum_{i=1}^{k} x_i \cdot L_i(x)$$

Then it generates two random values r, s to randomize the proof to have zero knowledge. The proof will be made of the following:

$$\begin{aligned}
A &= [\alpha + r\delta + A(\tau)]_1 \\
B &= [\beta + s\delta + B(\tau)]_2 \\
C &= [\delta^{-1}(L_w(\tau) + H(\tau) \cdot Z(\tau)) \\
&\quad + s(\alpha + r\delta + A(\tau)) \\
&\quad + r(\beta + s\delta + B(\tau)) \\
&\quad - rs\delta]_1
\end{aligned}$$

We can right away ask ourselves the question: Is the prover fine with sending this? Is the witness protected? There are a few things to consider:

- If we had just sent $[A(\tau)]_1$ (or $[B(\tau)]_1$ for that matter) over, this would not be zero-knowledge as it's not random and is leaking information about the value of $A(\tau)$. We need the r and s terms to make the whole expression random to the verifier. This is also secure because we are just sending an encryption of the expression and not the expression itself, and this is hard to break assuming CDH.

- The same remark can be made for C. The terms for which r, s are factors become random noise because they are products of terms that are unrelated from each other. Therefore C appears random to the verifier.

2.6 Groth'16 Step 3: Verifying

The verifier is given a verifying key, a proof A, B, C. Let's remove all the random shenanigans (I'll say a word about these later) introduced by the prover to make things clear by setting $r = s = 1$ and assume that:

$$\begin{aligned} A &= [\alpha + \delta + A(\tau)]_1 \\ B &= [\beta + \delta + B(\tau)]_2 \\ C &= [\delta^{-1}(L_w(\tau) + H(\tau) \cdot Z(\tau)) \\ &\quad + (\alpha + \delta + A(\tau)) \\ &\quad + (\beta + \delta + B(\tau))]_1 \end{aligned}$$

The final verifying check is of the form:

$$e(A, B) \stackrel{?}{=} e([\alpha]_1, [\beta]_2) + e([L_p(\tau)]_1, [\gamma]_2) + e(C, [\delta]_2)$$

We can do a quick completeness check by expanding both sides of the equation to make sure that if both parties are honest, then this check should pass.

Then, instead of giving a full soundness proof, we can provide helpful intuition as to why a dishonest prover will have a hard time forging a proof. This is not trivial. There is a lot of canceling of terms happening which can get confusing. Let's do it step by step:

- The second pairing is purely there to enforce that the prover has used the correct public inputs. To do so, we only ask it to compute the polynomial C (from the QAP) on the witnesses and we do it ourselves on the public inputs. At the end, by adding the pairings for C and the second pairing will give us the full polynomial. Now, assuming that the prover has used the correct public inputs, we can remove the second pairing altogether and assuming that C contains the full polynomial going forward.

- The third pairing is there so that the prover cannot just give us an arbitrary polynomial, but only something of the form $L(\tau) + H(\tau)Z(\tau)$ because it needs to use a product of δ^{-1} to not just give us a random value.

- The first pairing is there to balance the equations by providing the $\alpha\beta$ term needed.

This polynomial check at a random point, as well as the hardness of DLOG, and of an adjacent problem on the bilinear pairings that we used, should be enough to convince us with very high probability that the proof given is correct. It is also possible to prove the existence extractability of a witness if a proof is accepted, providing knowledge soundness.

Et voilà! We have conquered Groth'16 and gotten a taste of the SNARK!

2.7 Problem

1. Assume that δ was not destroyed properly and that a prover has access to it. Show that it can forge arbitrary proofs with it.

2. Show that Groth'16 is malleable, that is, that given a proof (A, B, C) and arbitrary values t, u, the triplet $(t^{-1}A, tB + [tu\delta]_1, C + uA)$ is a valid proof.

2.8 Resources to Learn More

1. You can use Groth'16 yourself! Look at Circom 2, for a good circuit compiler written in Rust.

2. The original paper Jens Groth (2016) is rigorous but not too inaccessible.